

Taller: Complejidad Algorítmica y Arreglos

Asignatura: Estructura de Datos (SD1004)

Docente: Juan Duque

Institución: Institución Universitaria Pascual Bravo

Semestre: 2026-11

Temas: Big O (Clase 2) | Arreglos 1D (Clase 3) | Arreglos 2D (Clase 4)

PARTE 1: COMPLEJIDAD ALGORÍTMICA (BIG O)

1.1 Clasificación de complejidad

1. Buscar una palabra en un diccionario físico dividiendo por la mitad el bloque de páginas restante en cada intento.

Complejidad: $O(\log n)$

Explicación: Cada vez que abres el libro justo en la mitad, la cantidad de páginas que te quedan por revisar se reduce a un 50%. Como vas descartando la mitad del problema de un solo golpe, el número de pasos no crece directo con las páginas, sino de forma logarítmica.

2. Revisar una por una las tarjetas de identificación en un contenedor hasta dar con la correcta.

Complejidad: $O(n)$

Explicación: Si tienes mala suerte, la tarjeta que buscas va a estar al final y te va a tocar mirar todas las tarjetas. Si duplicas la cantidad de carpetas o fichas en el contenedor, el tiempo de búsqueda se va a duplicar en la misma proporción.

3. Tomar la carta que quedó arriba en un mazo que ya fue mezclado.

Complejidad: $O(1)$

Explicación: Agarrar la primera carta requiere un solo movimiento directo. Da igual si el mazo tiene 10 cartas o 10.000; el esfuerzo físico e inmediato es exactamente el mismo.

4. Cruzar la fecha de cumpleaños de cada estudiante de un grupo contra las de todos los demás integrantes.

Complejidad: $O(n^2)$

Explicación: Cada persona del salón debe compararse individualmente contra los demás compañeros. Al ser una combinación cruzada de todos contra todos, el volumen de comparaciones sube al cuadrado del total de alumnos ($n \times n$).

5. Echarle una mirada a la hora que marca el reloj de pulsera.

Complejidad: $O(1)$

Explicación: Mirar la hora es una acción instantánea de un solo paso; no depende de ningún volumen de información ni de listas externas.

1.2 Aplicación práctica

Tarea cotidiana con comportamiento $O(n)$: Organizar a mano una fila de libros en una estantería limpiando el polvo de la portada uno por uno. Si tengo n libros, debo tomar cada uno, limpiarlo y colocarlo en su sitio.

Escenario si la entrada aumenta 10 veces: Si en vez de limpiar 8 libros me entregan 80, el proceso me tomará exactamente 10 veces más tiempo, ya que debo aplicar el mismo procedimiento lineal a cada libro.

1.3 Criterio de escalabilidad

Comparativa entre Forma A $O(n^2)$ y Forma B $O(n \log n)$:

Elección recomendada: Forma B ($O(n \log n)$).

Justificación: Aunque con $n = 5$ la opción A responde rápido por temas de sobrecarga inicial, la curva de $O(n^2)$ explota en cuanto aumentan los datos (con 1.000 datos requeriría 1.000.000 de operaciones). En cambio, la Forma B mantiene un crecimiento moderado (unas 10.000 operaciones para el mismo caso), garantizando que el sistema no se colapse al procesar grandes volúmenes.

1.4 Evaluación de afirmaciones

a. Un algoritmo $O(1)$ siempre es más rápido en tiempo real que uno $O(n)$.

Falso. La notación Big O define la tendencia de crecimiento, no la velocidad absoluta. Si la constante de la operación $O(1)$ es muy pesada (por ejemplo, pausar el sistema 5 segundos), un algoritmo $O(n)$ para un valor pequeño como $n = 2$ finalizará mucho antes.

b. La notación Big O describe cómo crece el tiempo de ejecución según el volumen de datos y no la duración en segundos.

Verdadero. Evalúa cómo escala la cantidad de trabajo respecto a n , independientemente del procesador o del reloj del equipo.

c. Consultar por índice en un arreglo (arreglo[5]) es $O(n)$ porque hay que pasar por los primeros 5 elementos.

Falso. El acceso por índice es $O(1)$. El equipo calcula directamente la posición exacta en memoria usando una fórmula aritmética simple, evitando recorrer las casillas previas.

PARTE 2: ARREGLOS UNIDIMENSIONALES (1D)

2.1 Estructura del arreglo

1. Representación visual de las calificaciones:

Estructura de Arreglo 1D: notas

Valor:	3.5	4.2	2.8	5.0	3.9	4.5
Índice:	[0]	[1]	[2]	[3]	[4]	[5]

2. Valor e índice de la tercera nota ingresada:

En la secuencia de datos ingresados, la tercera nota es 2.8 y queda almacenada en el índice [2] (recordando que el conteo inicia en cero).

3. Instrucción para acceder al último registro sin iterar:

```
nota_ultimo = notas[5]
```

2.2 Cálculo de direcciones en memoria

1. Posiciones de memoria para cada índice (Base = 0x2000, 4 bytes/elemento):

- $\text{notas}[0] = 0x2000 + (0 \times 4) = 0x2000$
- $\text{notas}[3] = 0x2000 + (3 \times 4) = 0x2000 + 12 \text{ (en hex)} = 0x200C$
- $\text{notas}[5] = 0x2000 + (5 \times 4) = 0x2000 + 20 \text{ (en hex)} = 0x2014$

2. Explicación del acceso directo $O(1)$:

Consultar `notas[3]` o `notas[0]` exige el mismo tiempo porque el procesador simplemente aplica la fórmula de multiplicación y suma dirección base + (índice × tamaño) en un solo ciclo de reloj para saltar a la dirección física de la memoria RAM.

2.3 Métodos de búsqueda

1. Búsqueda lineal para encontrar el valor 31:

- Paso 1: `edades[0] == 31?` (15 != 31)
- Paso 2: `edades[1] == 31?` (18 != 31)
- Paso 3: `edades[2] == 31?` (20 != 31)
- Paso 4: `edades[3] == 31?` (23 != 31)
- Paso 5: `edades[4] == 31?` (27 != 31)
- Paso 6: `edades[5] == 31?` (31 == 31) ¡Encontrado!

Total de comparaciones realizadas: 6

2. Búsqueda binaria para encontrar el valor 31:

- Paso 1: Punto medio entre 0 y 7 $\rightarrow \text{int}((0+7)/2) = 3$ (valor 23). Se evalúa $31 > 23$, por lo que se descarta la mitad izquierda y se busca del índice 4 al 7.
- Paso 2: Punto medio entre 4 y 7 $\rightarrow \text{int}((4+7)/2) = 5$ (valor 31). Se evalúa $31 == 31$ ¡Encontrado!

Total de comparaciones realizadas: 2

3. Requisito de ordenamiento en búsqueda binaria:

Funciona únicamente si la colección está ordenada porque se apoya en la lógica de descartar toda una mitad del arreglo comparando contra el centro. Si estuviera desordenado, no se podría saber si el elemento buscado está a la izquierda o a la derecha.

4. Selección para 1.000.000 de registros:

Se debe utilizar la búsqueda binaria. Mientras que la búsqueda lineal ($O(n)$) podría requerir hasta 1.000.000 de iteraciones, la binaria ($O(\log n)$) resuelve el hallazgo en un máximo de aproximadamente 20 comparaciones ($\log_2(1.000.000) \approx 20$).

2.4 Costo de inserción

1. Insertar al final:

- Con espacio libre: Es una operación de costo constante $O(1)$, basta con asignar el valor en la posición que sigue.
- Sin espacio libre: Tiene un costo lineal $O(n)$, ya que exige reservar una nueva estructura con mayor capacidad y duplicar elemento por elemento todo el contenido previo.

2. Insertar el valor 21 en la posición correspondiente para conservar el orden:

Es de costo $O(n)$. Es necesario desplazar hacia la derecha los datos mayores (23 y 27) para abrir la vacante donde se asignará el 21.

3. Resumen en notación Big O:

- Insertar al final con espacio: $O(1)$
- Insertar intermedio o con realojamiento de memoria: $O(n)$

PARTE 3: ARREGLOS BIDIMENSIONALES (2D)

3.1 Estructura matricial

1. Diagrama de la matriz de asientos de un aula (3 filas x 4 columnas):

Matriz Bidimensional: salón[3][4]

	Columna 0	Columna 1	Columna 2	Columna 3
Fila 0	salón[0][0]	salón[0][1]	salón[0][2]	salón[0][3]
Fila 1	salón[1][0]	salón[1][1]	salón[1][2]	salón[1][3]
Fila 2	salón[2][0]	salón[2][1]	salón[2][2]	salón[2][3]

2. Instrucción de acceso a la fila 2, columna 3:

estado = salon[2][3]

3. Capacidad total y regla general:

Esta matriz contiene $3 \times 4 = 12$ casillas.

Fórmula general: Total de elementos = Filas $\times$ Columnas.

3.2 Aplanamiento en memoria RAM (Row-Major Order)

1. Disposición de casillas en el direccionamiento continuo de la RAM:

Mapeo Contiguo en Memoria RAM (Row-Major Order)

Celda:	[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]	[2][0]	[2][1]	[2][2]
Dir:	0x1000	0x1004	0x1008	0x100C	0x1010	0x1014	0x1018	0x101C	0x1020	0x1024	0x1028

2. Dirección de memoria para la casilla salon[1][2] (Base = 0x1000, 4 bytes/elemento):

Fórmula: dirección = base + ((fila $\times$ columnas) + columna) $\times$ tamaño

Cálculo: $0x1000 + ((1 \times 4) + 2) \times 4 = 0x1000 + (6) \times 4 = 0x1000 + 24 \text{ bytes} \rightarrow 0x1018$

3. Direcciones para la primera y última casilla:

- $\text{salon}[0][0] = 0x1000 + ((0 \times 4) + 0) \times 4 = 0x1000$
- $\text{salon}[2][3] = 0x1000 + ((2 \times 4) + 3) \times 4 = 0x1000 + (11) \times 4 = 0x1000 + 44 \text{ bytes} \rightarrow 0x102C$

3.3 Recorrido con bucles anidados

1. Algoritmo para contar puestos ocupados:

ocupados = 0

Para f desde 0 hasta filas - 1 Hacer

 Para c desde 0 hasta columnas - 1 Hacer

 Si $\text{salon}[f][c] == 1$ Entonces

 ocupados = ocupados + 1

 FinSi

 FinPara

FinPara

Mostrar ocupados

2. Complejidad del algoritmo de conteo:

Es de orden $O(N)$, donde N es la cantidad total de casillas de la matriz ($N = f \times c$). Esto se debe a que el proceso recorre de manera individual cada celda una única vez.

3. Impacto si se duplican filas y columnas simultáneamente:

Si las filas se multiplican por 2 ($2f$) y las columnas por 2 ($2c$), el total de casillas pasa a ser $2f \times 2c = 4fc$. Por lo tanto, el número de iteraciones y el tiempo de ejecución aumentarán 4 veces.

3.4 Integración de conceptos

Big O, los arreglos 1D y las matrices 2D están totalmente conectados porque la forma en que organizamos los datos define la velocidad de nuestros programas. Entender Big O nos sirve para elegir la estructura adecuada según lo que necesitemos hacer: un arreglo 1D nos permite saltar a cualquier casilla en tiempo constante $O(1)$ si conocemos su posición, pero buscar un dato sin orden toma un tiempo lineal $O(n)$, lo cual nos obliga a ordenar la información para aplicar una búsqueda binaria en $O(\log n)$. Por su parte, los arreglos 2D nos ayudan a modelar problemas con cuadrículas o tablas, pero al recorrerlos con ciclos anidados el costo escala con el total de elementos. En conclusión, conocer cómo la memoria guarda las casillas de forma continua nos permite anticipar el rendimiento de los algoritmos y tomar mejores decisiones al programar.